Intrinisically Typed Syntax That Works (Fresh Perspective)

ANONYMOUS AUTHOR(S)

In dependently typed proof assistants, intrinsically typed syntax promises elegant mechanizations: well-typed terms are represented by well-typed data, substitution preserves typing by construction, and type safety proofs collapse to straightforward induction. For simple calculi such as the simply-typed lambda calculus this promise is fulfilled. For realistic polymorphic systems, however, the technique can become impractical: renamings and substitutions indexed by other substitutions force proofs to transport terms across equal types that are not definitionally equal. This phenomenon—informally known as transfer hell—causes even routine metatheoretic arguments to be dominated by administrative reasoning.

This fresh perspective shows how to make intrinsically typed syntax work in practice for polymorphic calculi. We show that giving substitutions canonical computational behavior (via Agda rewriting) yields a mechanization of System F's metatheory in Agda that avoids explicit transport reasoning.

Our approach simplifies proofs of metatheoretic properties: typing preservation becomes definitional, substitution lemmas and corresponding transfers disappear. Generally, when intrinsic encodings generate transports, the right move is often to strengthen definitional equality rather than add more lemmas. We extract a practical recipe for applying this methodology.

ACM Reference Format:
Anonymous Author(s). 2018. Intrinisically Typed Syntax That Works (Fresh Perspective). *ACM Trans. Program. Lang. Syst.* 37, 4, Article 111 (August 2018), 17 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Intrinsically typed syntax is widely recommended for mechanizing type systems in dependently typed proof assistants. By representing well-typed terms as well-typed data, typing invariants hold by construction as in a Church-style presentation, so typing preservation is immediate. For small calculi this approach is highly effective. However, for realistic polymorphic calculi such as System F, mechanizations often become complicated: conceptually simple proofs become cluttered with transports due to administrative equalities.

The simply-typed lambda calculus is the canonical example where intrinsically typed syntax is particularly effective. Figure 1 shows typical Agda definitions of the syntax of types, type contexts, variables, and expressions, which are indexed by contexts and types (left column)[1]. Building on these definitions we can define a small-step operational semantics and prove type safety. This is because subject reduction holds by construction and progress follows by a simple inductive proof (right column). We elide the definition of the substitution operator _[_], as we discuss substitutions in detail in Section 2.1.

The intrinsic encoding makes type safety proofs so concise that they can serve as textbook examples [7]. Nothing comparable to transport reasoning arises in this development as only definitional properties of substitutions are exercised. However, the same approach breaks down for polymorphic calculi and the remainder of this paper explains how to recover some of its simplicity.

Transfer hell is the colloquial term for being forced to apply type-changing lemmas to terms that are already known to be equal. This phenomenon appears, for example, in intrinsic mechanizations of System F [2, 3, 12]: expression substitutions depend on type substitutions and type substitutions depend on renamings; composing them yields types that are equal only propositionally, forcing

[1]Symbols like _▷_ are mixfix operators where arguments replace the underlines in the symbols. Parameters in curly braces are implicit, which means they can be omitted if they are inferable.

2018. ACM 1558-4593/2018/8-ART111
https://doi.org/XXXXXXX.XXXXXXX

```
data Type : Set where
  𝟙    : Type
  _⇒_ : Type → Type → Type

data Ctx : Set where
  ∅    : Ctx
  _▷_ : Ctx → Type → Ctx

data _∋_ : Ctx → Type → Set where
  here  : (Γ ▷ T) ∋ T
  there : Γ ∋ T → (Γ ▷ U) ∋ T

data _⊢_ Γ : Type → Set where
  con : Γ ⊢ 𝟙
  var : Γ ∋ T → Γ ⊢ T
  lam : (Γ ▷ T) ⊢ U → Γ ⊢ (T ⇒ U)
  app : Γ ⊢ (T ⇒ U) → Γ ⊢ T → Γ ⊢ U

data _⟶_ {Γ} {T} : Γ ⊢ T → Γ ⊢ T → Set where
  β-lam : app (lam e₁) e₂ ⟶ (e₁ [ e₂ ])
  ξ-app : e₁ ⟶ e₁′ → app e₁ e₂ ⟶ app e₁′ e₂

data Value {Γ} : Γ ⊢ T → Set where
  con : Value con
  lam : (e : (Γ ▷ T) ⊢ U) → Value (lam e)

data Progress (e : Γ ⊢ T) : Set where
  done : Value e → Progress e
  step : e ⟶ e′ → Progress e

progress : (e : ∅ ⊢ T) → Progress e
progress con       = done con
progress (lam e)   = done (lam e)
progress (app e₁ e₂)
  with progress e₁
... | step x       = step (ξ-app x)
... | done (lam e) = step β-lam
```

Fig. 1. Call-by-name simply-typed lambda calculus (adapted from the PLFA textbook [7])

transports throughout proofs. The root problem is that substitutions that are propositionally equal are not definitionally equal, so terms must repeatedly be transported across equal types.

One workaround is to state definitions and proofs with explicit equivalence relations, but this practice is cumbersome, inefficient, and far away from the concision of definitional equality. Our proposal thus shifts the focus to computation on substitutions: if extensionally equal substitutions do not normalize identically, transport reasoning is unavoidable.

Hence, the simple message of this fresh perspective is: when intrinsically typed syntax becomes complicated, the problem is often not the encoding of terms but the definition of equality on substitutions. Strengthening definitional equality can simplify metatheoretical proofs more effectively than adding automation or additional lemmas.

We substantiate our claims with a case study on renaming and substitution in an intrinsic encoding of System F. We strengthen definitional equality by internalizing the equational theory of substitutions (the σ-calculus) using Agda's rewriting mechanism. As a consequence, extensionally equal substitutions normalize to the same form and transports disappear.

Contributions

- We show how to eliminate "transfer hell" in intrinsically typed encodings of System F by internalizing the equational laws of the σ-calculus for substitutions using Agda's rewriting.
- We demonstrate that the resulting rewrite system is compatible with Agda's definitional equality (i.e., terminating and confluent) for a functional representation of substitutions.
- We mechanize parts of the metatheory of System F, obtaining substantially simpler proofs that are more efficient to check.
- From this experience we extract practical guidelines for designing intrinsically typed syntax in dependently typed proof assistants that support rewriting.

```
data Type (n : Nat) : Set where
  ‘_    : Var n → Type n
  ∀α    : Type (1 + n) → Type n
  _⇒_ : Type n → Type n → Type n
```

```
_ : Type 0 - a closed type:  ∀α. α→α
_ = ∀α (‘ zero ⇒ ‘ zero)
_ : Type 0 - ∀αβ. α→β→α
_ = ∀α (∀α (‘ suc zero ⇒ ‘ zero ⇒ ‘ suc zero))
```

Fig. 2. Syntax of System F types (Agda definition left, examples right)

All code fragments in this document are extracted from typechecked Agda code, which is available as a supplement.

Related Work and Positioning

To our knowledge, previous intrinsically typed mechanizations of System F [2, 3, 12] follow a natural design: expression syntax is indexed by typing derivations and substitutions are indexed by substitutions at the type level. While elegant at the level of definitions, this design makes proofs brittle, because they are prone to transfer hell. Our work eliminates transfers resulting from substitutions, thus avoiding transfer hell.

Recent work studies Rewriting Type Theory (RTT) [5] and its implementations in Agda [4] and Rocq [8, 9]. RTT adds rewrite rules to dependent type theories by turning propositional equalities into reduction rules: a term matched by the left-hand side of an equation reduces to the right-hand side. Rewrites are safe if they preserve subject reduction and consistency. The former is guaranteed by confluence of the combined rewrite system, the latter by only rewriting proved propositional equalities. Our work is based on Agda's implementation of RTT.

Our approach is inspired by the work on Autosubst [13–15] and ultimately by Abadi et al. [1]. While Autosubst relies on tactics to normalize substitutions, we do so by rewriting. We build on a standard functional representation of renamings and substitutions [2]. The main challenge in setting up the σ-calculus is to fine-tune the reduction behavior between the definitions and the rewrite rules to ensure that the combined reduction relation remains terminating and confluent.

Overview

Section 2 presents our scoped encoding of System F types. It discusses the standard encoding of functional renamings and substitutions (Section 2.2) and introduces the σ-calculus laws (Section 2.3). Section 3 presents the intrinsically typed encoding of System F expressions. It defines expression substitutions (Section 3.2) and a small-step operational semantics (Section 3.3). Section 4 demonstrates a concrete instance of transfer hell, once treated manually and once by using our method. Section 5 outlines methodological issues along with recommendations and guidelines.

2 System F Types

2.1 Syntax

The syntax of System F types uses the standard de Bruijn encoding of variables by numbers. The type Type n (Figure 2) stands for the set of types with n type variables in scope. Henceforth, we call such n a *scope*. Type variables for scope n are drawn from the type Var n, i.e., the set $\{0, \ldots, n-1\}$, written as zero, suc zero, suc (suc zero), ...and ranged over by α. The $\forall\alpha$-binder introduces a new type variable, and $\Rightarrow$ is the infix function type constructor. We call this style of syntax *intrinsically scoped* as it can only express well-scoped types. We let T range over types.

```
-- renamings
Ren : Nat → Nat → Set
Ren n₁ n₂ = Var n₁ → Var n₂

opaque
  -- weakening
  wkᴿ : Ren n (1 + n)
  wkᴿ = suc

  -- identity renaming
  idᴿ : Ren n n
  idᴿ α = α

  -- extend with new variable
  _·ᴿ_ : Var n₂ → Ren n₁ n₂ → Ren (1 + n₁) n₂
  (α ·ᴿ ζ) zero    = α
  (_ ·ᴿ ζ) (suc α) = ζ α

  -- apply renaming to variable
  _&ᴿ_ : Var n₁ → Ren n₁ n₂ → Var n₂
  α &ᴿ ζ = ζ α

  -- left-to-right composition
  _⨟ᴿ_ : Ren n₁ n₂ → Ren n₂ n₃ → Ren n₁ n₃
  (ζ₁ ⨟ᴿ ζ₂) α = ζ₂ (ζ₁ α)

-- lifting
_↑ᴿ : Ren n₁ n₂ → Ren (1 + n₁) (1 + n₂)
_↑ᴿ ζ = zero ·ᴿ (ζ ⨟ᴿ wkᴿ)

-- apply renaming to type
_[_]ᴿ : Type n₁ → Ren n₁ n₂ → Type n₂
(` α) [ ζ ]ᴿ       = ` (α &ᴿ ζ)
(∀α T) [ ζ ]ᴿ      = ∀α (T [ ζ ↑ᴿ ]ᴿ)
(T₁ ⇒ T₂) [ ζ ]ᴿ = (T₁ [ ζ ]ᴿ) ⇒ (T₂ [ ζ ]ᴿ)
```

```
-- substitutions
Sub : Nat → Nat → Set
Sub n₁ n₂ = Var n₁ → Type n₂

opaque
  -- lift renaming to substitution
  ⟨_⟩ : Ren n₁ n₂ → Sub n₁ n₂
  ⟨ ζ ⟩ α = ` (α &ᴿ ζ)

  -- extend with new type
  _·ˢ_ : Type n₂ → Sub n₁ n₂ → Sub (1 + n₁) n₂
  (T ·ˢ η) zero    = T
  (T ·ˢ η) (suc α) = η α

  -- apply substitution to variable
  _&ˢ_ : Var n₁ → Sub n₁ n₂ → Type n₂
  α &ˢ η = η α

  -- lifting
  _↑ˢ : Sub n₁ n₂ → Sub (1 + n₁) (1 + n₂)
  _↑ˢ η = (` zero) ·ˢ λ α → (η α) [ wkᴿ ]ᴿ

-- apply substitution to type
_[_]ˢ : Type n₁ → Sub n₁ n₂ → Type n₂
(` α) [ η ]ˢ       = α &ˢ η
(∀α T) [ η ]ˢ      = ∀α (T [ η ↑ˢ ]ˢ)
(T₁ ⇒ T₂) [ η ]ˢ = (T₁ [ η ]ˢ) ⇒ (T₂ [ η ]ˢ)

opaque
  -- left-to-right composition
  _⨟ˢ_ : Sub n₁ n₂ → Sub n₂ n₃ → Sub n₁ n₃
  (η₁ ⨟ˢ η₂) α = (η₁ α) [ η₂ ]ˢ
```

Fig. 3. Renamings and substitutions on types

2.2 Renaming and Substitution

The most important operations on types are consistent renaming of type variables and substitution. Figure 3 contains definitions for renamings (left column) and substitutions (right column). We defer the discussion of using `opaque` to Section 2.3.

Table 1 summarizes the operator layer used throughout the development. The table is intended as a reference map: renaming operators $\zeta : \mathsf{Ren}\ n_1\ n_2$ capture structure-preserving variable reindexing from scope n_1 to scope n_2, while substitution operators $\eta : \mathsf{Sub}\ n_1\ n_2$ perform capture-avoiding substitution of types from scope n_2 for variables in a type at scope n_1. Rather than introducing each symbol ad hoc in later proofs, we fix this interface once and reuse it uniformly.

Family	Operator	Role	Status
Renaming	wk^R	weakening renaming	O
Renaming	id^R	identity renaming	O
Renaming	$\alpha \cdot^R \zeta$	extend renaming with head variable α	O
Renaming	$\alpha \mathbin{\&}^R \zeta$	apply renaming to a variable	O
Renaming	$\zeta_1 \fatsemi^R \zeta_2$	compose renamings	O
Renaming	$\zeta \uparrow^R$	lift renaming under a binder	U
Renaming	$\mathsf{T}\ [\ \zeta\ ]^R$	apply renaming to a type	U
Substitution	$\langle\ \zeta\ \rangle$	embed renaming as substitution	O
Substitution	$\mathsf{T} \cdot^S \eta$	extend substitution with head type T	O
Substitution	$\alpha \mathbin{\&}^S \eta$	apply substitution to a variable	O
Substitution	$\eta_1 \fatsemi^S \eta_2$	compose substitutions	O
Substitution	$\eta \uparrow^S$	lift substitution under a binder	O
Substitution	$\mathsf{T}\ [\ \eta\ ]^S$	apply substitution to a type	U

Table 1. Operator inventory for type renamings and substitutions. Status: O for opaque, U for unfolding

Two design choices are important for what follows. First, renamings and substitutions have a standard functional representation [2, 7, 11], which keeps composition and lifting explicit. Second, the operators in this interface are the symbols on which we later impose the σ-calculus equations as rewrite rules. This separation between interface and equations enables us to control reduction behavior precisely.

2.3 The σ-Calculus with First-class Renamings

This section presents the equalities of the σ-calculus. These are the equalities that we *actually* use for computing with renamings and substitutions, rather than their defining equations in Figure 3. The opaque blocks control this behavior. Function symbols defined in an opaque block are usable outside the block, but they do not reduce. The only reducing definitions in the renaming block are $\zeta \uparrow^R$ for lifting a renaming and application of a renaming to a type. In the substitution block, only the application of a substitution to a type reduces.

This selective reduction behavior is essential for our approach because we want to compute using the laws of the σ-calculus with first-class renamings on top of the thus defined symbols, rather than their definitions. Blocking their reduction is necessary because the equalities from σ-calculus interfere adversely with the standard definitional equalities (see Section 5.1 for an example). The original defining equalities are available for proofs: We can open the opaque definitions locally and proceed as usual.

Next we introduce equational laws of the σ-calculus with first-class renamings [13]. All of these equalities are supported by proofs using the definitions in Figure 3[2].

The first group presents computation rules for substitutions.

$$
\begin{aligned}
&\mathsf{beta\text{-}ext\text{-}zero} : \mathsf{zero} \mathbin{\&}^S (T \cdot^S \eta) &&\equiv T\\
&\mathsf{beta\text{-}ext\text{-}suc} : \mathsf{suc}\ \alpha \mathbin{\&}^S (T \cdot^S \eta) &&\equiv \alpha \mathbin{\&}^S \eta\\
&\mathsf{beta\text{-}rename} : \alpha \mathbin{\&}^S \langle\ \zeta\ \rangle &&\equiv\ \text{`}\ (\alpha \mathbin{\&}^R \zeta)\\
&\mathsf{beta\text{-}comp} : \alpha \mathbin{\&}^S (\eta_1 \fatsemi^S \eta_2) &&\equiv (\alpha \mathbin{\&}^S \eta_1)\ [\ \eta_2\ ]^S\\
&\mathsf{beta\text{-}lift} : \eta \uparrow^S &&\equiv (\text{`}\ \mathsf{zero}) \cdot^S (\eta \fatsemi^S \langle\ \mathsf{wk}^R\ \rangle)
\end{aligned}
$$

Starting with the application of a substitution to a variable, there is one equation for each possible form of a substitution: extended substitution, lifted renaming, and composition of substitutions. The

[2]All proofs are available in the supplement.

final equation in this group characterizes lifting in terms of extension, weakening, and composition. It is *not* possible to use this equation to define lifting in Figure 3 as it would lead to circularity.

The second group contains laws about interactions between substitutions.

$$
\begin{array}{lll}
\text{associativity} & : (\eta_1 \fatsemi^S \eta_2) \fatsemi^S \eta_3 & \equiv \eta_1 \fatsemi^S (\eta_2 \fatsemi^S \eta_3) \\
\text{distributivity} & : (T \cdot^S \eta_1) \fatsemi^S \eta_2 & \equiv (T\,[\,\eta_2\,]^S) \cdot^S (\eta_1 \fatsemi^S \eta_2) \\
\text{interact} & : \langle\, \mathsf{wk}^R \,\rangle \fatsemi^S (T \cdot^S \eta) & \equiv \eta \\
\text{comp-id}_r & : \eta \fatsemi^S \langle\, \mathsf{id}^R \,\rangle & \equiv \eta \\
\text{comp-id}_l & : \langle\, \mathsf{id}^R \,\rangle \fatsemi^S \eta & \equiv \eta \\
\eta\text{-id} & : (\text{`}\,\mathsf{zero}\ \{n\}) \cdot^S \langle\, \mathsf{wk}^R \,\rangle & \equiv \langle\, \mathsf{id}^R \,\rangle \\
\eta\text{-law} & : (\mathsf{zero}\ \&^S\ \eta) \cdot^S (\langle\, \mathsf{wk}^R \,\rangle \fatsemi^S \eta) & \equiv \eta
\end{array}
$$

Composition is associative and it distributes over extension. First weakening and then extending with T has no effect, as T is substituted for a non-existing variable. The identity substitution is a right and left identity. Two η-laws about the interaction of extension and weakening conclude this group.[3]

There are analogous rules for renamings corresponding to the first two groups. These rules are omitted for space reasons.

The third group presents laws that govern the composition of different kinds of term traversal, that is, the behavior of the identity renaming and the four possible compositions of renamings and substitutions.

$$
\begin{array}{lll}
\text{identity}_r & : T\,[\,\mathsf{id}^R\,]^R & \equiv T \\
\text{compositionality}^{RS} & : (T\,[\,\zeta_1\,]^R)\,[\,\eta_2\,]^S & \equiv T\,[\,\langle\,\zeta_1\,\rangle \fatsemi^S \eta_2\,]^S \\
\text{compositionality}^{RR} & : (T\,[\,\zeta_1\,]^R)\,[\,\zeta_2\,]^R & \equiv T\,[\,\zeta_1 \fatsemi^R \zeta_2\,]^R \\
\text{compositionality}^{SR} & : (T\,[\,\eta_1\,]^S)\,[\,\zeta_2\,]^R & \equiv T\,[\,\eta_1 \fatsemi^S \langle\,\zeta_2\,\rangle\,]^S \\
\text{compositionality}^{SS} & : (T\,[\,\eta_1\,]^S)\,[\,\eta_2\,]^S & \equiv T\,[\,\eta_1 \fatsemi^S \eta_2\,]^S
\end{array}
$$

The final group contains laws that mediate between substitutions and renamings. Substitution of a lifted renaming is a renaming; the composition of two lifted renamings is the lifting of their composition as renamings.

$$
\begin{array}{lll}
\text{coincidence} & : T\,[\,\langle\,\zeta\,\rangle\,]^S & \equiv T\,[\,\zeta\,]^R \\
\text{coincidence-comp} & : \langle\,\zeta_1\,\rangle \fatsemi^S \langle\,\zeta_2\,\rangle & \equiv \langle\,\zeta_1 \fatsemi^R \zeta_2\,\rangle
\end{array}
$$

In general, the extraction of renamings from substitutions requires a dedicated solving strategy [15] and we have omitted some further coincidence laws for brevity.

Orienting these equations from left to right results in a confluent rewrite system for the σ-calculus that we install in Agda's computation engine using the REWRITE pragma [4]. Confluence of these rewrite equations is mechanically checked by Agda using the flags `--rewriting --confluence-check`. We discuss safety considerations for rewriting in Section 5.1. From now on, all equations shown in this section Section 2.3 are treated as definitional equalities.

As a consistency check for the definitions, we prove in Figure 4 that lifting a substitution and the action of a substitution satisfy the functor laws. With σ-calculus rewriting installed, all these lemmas hold definitionally as evidenced by their proof using reflexivity (refl). The lemmas marked with $*$ correspond directly to laws from the σ-calculus.[4] The lifting laws are shown on the left, the application laws on the right. The analogous results for renamings follow exactly the same pattern.

[3] In the statement of η-id, we write zero $\{n\}$ to tell Agda's type checker that n is the scope of type variable zero. This notation supplies an *implicit argument* that Agda can usually infer.

[4] The naming of the lemmas is taken from Chapman et al. [3] to enable direct comparison.

```
lifts*-id : (id^S {n} ↑^S) ≡ id^S
lifts*-id = refl

lifts*-comp : (η′ ⨟^S η) ↑^S ≡ (η′ ↑^S) ⨟^S (η ↑^S)
lifts*-comp = refl

sub*-id : T [ id^S ]^S ≡ T
sub*-id = refl

sub*-var : (` α) [ η ]^S ≡ α &^S η
sub*-var = refl  - *

sub*-comp : T [ η ⨟^S η′ ]^S ≡ (T [ η ]^S) [ η′ ]^S
sub*-comp = refl - *
```

Fig. 4. Functor laws for substitution proved

```
weaken : Type n → Type (1 + n)
weaken T = T [ wk^R ]^R

_[_]* : Type (1 + n) → Type n → Type n
T [ T′ ]* = T [ T′ ·^S id^S ]^S

data Ctx : Nat → Set where
  ∅    : Ctx zero
  _▷_  : Ctx n → Type n → Ctx n
  _▷*  : Ctx n → Ctx (1 + n)

data _∋_ : Ctx n → Type n → Set where
  zero : (Γ ▷ T) ∋ T
  suc  : Γ ∋ T → (Γ ▷ T′) ∋ T
  suc* : Γ ∋ T → (Γ ▷*) ∋ weaken T

data Expr (Γ : Ctx n) : Type n → Set where
  `_   : Γ ∋ T →
         Expr Γ T
  λx   : Expr (Γ ▷ T₁) T₂ →
         Expr Γ (T₁ ⇒ T₂)
  _·_  : Expr Γ (T₁ ⇒ T₂) →
         Expr Γ T₁ →
         Expr Γ T₂
  Λα   : Expr (Γ ▷*) T →
         Expr Γ (∀α T)
  _·*_ : Expr Γ (∀α T) →
         (T′ : Type n) →
         Expr Γ (T [ T′ ]*)
```

Fig. 5. Type contexts, variables, and expressions

3 System F Expressions

3.1 Syntax

Figure 5 defines the ingredients of intrinsically typed expressions: type weakening (weaken) and single substitution T [T′]* of a type T' for the innermost variable of T along with type contexts Γ, expression variables x, and expression syntax e. These ingredients provide the infrastructure for defining substitution and semantics in the following subsections.

Type contexts and expression variables follow the work of Chapman et al. [3]. A type context Γ : Ctx n tracks expression and type variables in scope n. The operator _▷_ adds a value binding while _▷* adds a type binding to Γ. Correspondingly, expression variables of type Γ ∋ T are implemented as typed de Bruijn indices with an extra case suc* to skip over type variables in the context.

Skipping with suc* requires weakening of the type as it is used under one more type binding. The lambda expression adds a value binding, while the type lambda adds a type binding. Type application requires the single substitution.

Expressions e : Expr Γ T are indexed by a context Γ and a type T with the intended reading as a typing judgment Γ ⊢ e : T. As customary in an intrinsically typed setting, each expression constructor corresponds to a typing rule in System F. Figure 6 presents some example expressions

```
ℕᶜ : Type ∅ *                          zeroᶜ : Expr ∅ ℕᶜ
ℕᶜ = ∀α ((‘α ⇒ ‘α) ⇒ (‘α ⇒ ‘α))        zeroᶜ = Λα (λg (λx ‘x))

oneᶜ : Expr ∅ ℕᶜ                       twoᶜ : Expr ∅ ℕᶜ
oneᶜ = Λα (λg (λx (‘g · ‘x)))          twoᶜ = succᶜ · (succᶜ · zeroᶜ)

succᶜ : Expr ∅ (ℕᶜ ⇒ ℕᶜ)
succᶜ = λx (Λα (λg (λx (‘g · ((((‘ (suc (suc (suc* zero)))) ·* ‘α) · ‘g) · ‘x)))))
```

Fig. 6. Church numerals in System F

and types. For readability, we have aliases ‘α for ‘ Z, ‘β for ‘ (S Z), and analogously for expression variables and binders.

3.2 Renaming and Substitution

The definition of a small-step operational semantics requires expression renamings and substitution to implement beta reduction for value abstractions and type abstractions. Renamings and substitutions are both indexed by type renamings and type substitutions, respectively.

Expression renamings ρ are indexed by type renamings ζ. As usual, they map a variable from one context to another. We write $\zeta \mid \ldots$ for any operation on renamings that is indexed by type renaming ζ. We only show the following representative definition as an example. The remaining operations may be found in the supplement.

```
_|_⇒ᴿ_ : Ren n₁ n₂ → Ctx n₁ → Ctx n₂ → Set
ζ | Γ₁ ⇒ᴿ Γ₂ = ∀ T → (x : Γ₁ ∋ T) → Γ₂ ∋ (T [ ζ ]ᴿ)
```

Expression substitutions σ are indexed by type substitutions η. A substitution takes a variable x of type T in context Γ_1 and maps it to an expression typed in context Γ_2 adjusting its type according to the type substitution: $T[\eta]^S$. The notation is analogous to the one for renamings.

```
_|_⇒ˢ_ : Sub n₁ n₂ → Ctx n₁ → Ctx n₂ → Set
η | Γ₁ ⇒ˢ Γ₂ = ∀ T → (x : Γ₁ ∋ T) → Expr Γ₂ (T [ η ]ˢ)
```

Applying a substitution to a variable corresponds to function application. Nevertheless, we use the same interface as for type substitutions to enable using the same rewriting machinery on the expression level again. The following definition includes the typing of all symbols. Subsequently, we omit all types that can be inferred.

```
_|_&ˢ_ : ∀ {Γ₁ : Ctx n₁} {Γ₂ : Ctx n₂} (η : Sub n₁ n₂)
  → (x : Γ₁ ∋ T) → (σ : η | Γ₁ ⇒ˢ Γ₂) → Expr Γ₂ (T [ η ]ˢ)
η | x &ˢ σ = σ _ x
```

The identity substitution for expressions appears simple only because of the installed rewriting of type substitutions. Without the rewriting, the right-hand side's type is T, but the expected type is $T[\mathsf{id}^s]^S$, which would require the transport lemma comp-id$_r$.

```
Idˢ : idˢ | Γ ⇒ˢ Γ
Idˢ _ = λ x → ‘ x
```

Extending a substitution is standard: given an expression e and a substitution σ, we return a substitution that sends variable zero to e and otherwise resorts to σ.

```
_|_·ˢ_ : ∀ η → (e : Expr Γ₂ (T [ η ]ˢ)) → (σ : η | Γ₁ ⇒ˢ Γ₂) → η | (Γ₁ ▷ T) ⇒ˢ Γ₂
(_ | e ·ˢ σ) _ zero    = e
(_ | e ·ˢ σ) _ (suc x) = σ _ x
```

Due to the structure of contexts, we need two lifting lemmas when defining the application of a substitution to an expression, one for value bindings and another for type bindings. In both cases, the substitution σ needs to be adjusted (i.e., lifted) to accommodate the newly introduced binding. The former is standard: it maps the newly introduced variable to itself (zero) and weakens the images of σ accordingly. However, the definition of lifting requires a transport lemma to line up the type substitutions, which is applied by rewriting. The last part of the definition $\mathsf{id}^R \mid \ldots \mathsf{Wk}^R \ldots$ applies an expression renaming (weakening by $T[\eta]^S$) indexed by id^R to the output of σ.

```
_|_⇑ˢ_ : ∀ η → (σ : η | Γ₁ ⇒ˢ Γ₂) → ∀ T → η | (Γ₁ ▷ T) ⇒ˢ (Γ₂ ▷ (T [ η ]ˢ))
η | σ ⇑ˢ T = η | (` zero) ·ˢ λ _ x → idᴿ | (σ _ x) [ Wkᴿ (T [ η ]ˢ) ]ᴿ
```

Lifting a substitution over a type binding is non-standard. The type substitution requires lifting, but no rewriting is needed.

```
_|_⇑ˢ* : ∀ η → (σ : η | Γ₁ ⇒ˢ Γ₂) → (η ↑ˢ) | (Γ₁ ▷*) ⇒ˢ (Γ₂ ▷*)
(η | σ ⇑ˢ*) = (η ⨟ˢ ⟨ wkᴿ ⟩) | (` zero) ·ˢ* λ _ x → wkᴿ | (σ _ x) [ wkᴿ* ]ᴿ
```

Substitution application is defined by structural recursion on expressions. The cases dispatch to the operations that we introduced: applying a substitution to a variable, lifting over a value binding (case λx), lifting over a type binding (case $\Lambda\alpha$), the cases for application and type-application are homomorphic.

```
_|_[_]ˢ : (η : Sub n₁ n₂) → (e : Expr Γ₁ T) → (σ : η | Γ₁ ⇒ˢ Γ₂) → Expr Γ₂ (T [ η ]ˢ)
η | (` x) [ σ ]ˢ     = η | x &ˢ σ
η | (λx e) [ σ ]ˢ    = λx (η | e [ η | σ ⇑ˢ _ ]ˢ)
η | (Λα e) [ σ ]ˢ    = Λα ((η ↑ˢ) | e [ η | σ ⇑ˢ* ]ˢ)
η | (e · e₁) [ σ ]ˢ  = (η | e [ σ ]ˢ) · (η | e₁ [ σ ]ˢ)
η | (e ·* T) [ σ ]ˢ = (η | e [ σ ]ˢ) ·* (T [ η ]ˢ)
```

3.3 Semantics

We are now ready to define a small-step operational semantics. As in the introduction, we opt for call-by-name for concision. The definition of single substitution, needed for beta reduction, is analogous to the one on types: we build the substitution by extending the identity substitution with the argument e'. As usual in an intrinsically typed setting, this definition already proves that substitution preserves typing.

```
_[_] : Expr (Γ ▷ T′) T → Expr Γ T′ → Expr Γ T
e [ e′ ] = idˢ | e [ idˢ | e′ ·ˢ Idˢ ]ˢ
```

In addition, we have to substitute a type into an expression to cater for beta reduction of type applications. This substitution is also implemented by an expression substitution indexed by a type substitution that performs the actual change.

```
_[*_*] : Expr (Γ ▷*) T → (T′ : Type n) → Expr Γ (T [ T′ ]*)
e [* T′ *] = (T′ ·ˢ idˢ) | e [ idˢ | T′ ·ˢ* Idˢ ]ˢ
```

Due to the extended type substitution on the left, we also have to adapt the expression substitution on the right to the extended context structure. This adaptation is the type-level analogue to extension:

```
data Value : Expr Γ T → Set where
  λx : (e : Expr (Γ ▷ T₁) T₂) → Value (λx e)
  Λα : (v : Value e)          → Value (Λα e)

data Progress : Expr Γ T → Set where
  done : (v : Value e) → Progress e
  step : (e⟶e′ : e ⟶ e′) → Progress e

NoVar : Ctx n → Set
NoVar Γ = ∀ {T′} → ¬ (Γ ∋ T′)

progress : NoVar Γ → (e : Expr Γ T) → Progress e
progress nv (` x)   = ⊥-elim (nv x)
progress nv (λx e) = done (λx e)
progress nv (e · e₁)
  with progress nv e
... | done (λx e₂)   = step β-λ
... | step e⟶e′ = step (ξ-·₁ e⟶e′)
progress nv (Λα e)
  with progress (λ{ (suc* x) → nv x}) e
... | done v       = done (Λα v)
... | step e⟶e′ = step (ξ-Λ e⟶e′)
progress nv (e ·* T′)
  with progress nv e
... | done (Λα v) = step β-Λ
... | step e⟶e′ = step (ξ-·₁* e⟶e′)
```

Fig. 7. Progress for System F

```
_|_·ˢ*_ : ∀ η T → (σ : η | Γ₁ ⇒ˢ Γ₂) → (T ·ˢ η) | (Γ₁ ▷*) ⇒ˢ Γ₂
(_ | T ·ˢ* σ) _ (suc* x) = σ _ x
```

With substitution in place, we present the definition of the call-by-name small-step reduction relation. As usual, such a definition of reduction for intrinsically-typed syntax implies subject reduction by construction.

```
data _⟶_ : Expr Γ T → Expr Γ T → Set where
  β-λ : (λx e₁ · e₂)  ⟶ (e₁ [ e₂ ])
  β-Λ : (Λα e ·* T′) ⟶ (e [* T′ *])
  ξ-·₁  : e₁ ⟶ e₁′ → (e₁ · e₂) ⟶ (e₁′ · e₂)
  ξ-·₁* : e ⟶ e′   → (e ·* T)  ⟶ (e′ ·* T)
  ξ-Λ : e ⟶ e′   → (Λα e)  ⟶ (Λα e′)
```

To obtain type safety, it remains to prove progress. Figure 7 presents the complete proof of progress for call-by-name System F using the structure from PLFA [7]. A value is either a lambda or a type lambda where the body is already a value. An expression satisfies progress if it is either a value (done) or if it can make a step (step).

Due to the definition of type lambda values, we have to establish progress for contexts that may contain type bindings, but no value bindings. These contexts are characterized literally by the function NoVar.

The proof of progress is by induction on the structure of expressions. The overall structure of this proof closely follows Chapman et al [3]. Their proof is more complicated because their semantics is call-by-value, their language supports isorecursive types, and their subject language is System Fω.

Next, we move on to study an example of transfer hell in detail.

4 Transfer Heaven and Transfer Hell

In this section, we study functorial and monadic properties of expression substitutions. These properties are needed once we consider properties of logical relations [12].

As an example, we examine the composition of two expression substitutions. Its definition is reasonably compact, yet proving its functoriality showcases a significant amount of transfer hell. The definition involves two type substitutions η_1 and η_2 that index the corresponding expression substitutions σ_1 and σ_2. The composed expression substitution is indexed by the composition of η_1 and η_2.

```
_,_|_⨟ˢ_ : ∀ η₁ η₂ → (σ₁ : η₁ | Γ₁ ⇒ˢ Γ₂) → (σ₂ : η₂ | Γ₂ ⇒ˢ Γ₃) → (η₁ ⨟ˢ η₂) | Γ₁ ⇒ˢ Γ₃
(η₁ , η₂ | σ₁ ⨟ˢ σ₂) _ x = η₂ | (σ₁ _ x) [ σ₂ ]ˢ
```

In the following subsections, we state and prove functoriality of composition, once in heaven—with σ-calculus rewriting in place—and once in hell.

4.1 Transfer Heaven: Composition of Substitutions with Rewriting

The functoriality property of expression substitution says that applying two substitutions one after the other to an expression is the same as applying their composition. Without σ-calculus rewriting in place, the types of the left-hand side and right-hand side of the equality are different and would require a transfer.

```
Compositionality^SS : ∀ (e : Expr Γ₁ T) (η₁ : Sub n₁ n₂) (η₂ : Sub n₂ n₃)
  → (σ₁ : η₁ | Γ₁ ⇒ˢ Γ₂) (σ₂ : η₂ | Γ₂ ⇒ˢ Γ₃)
  → η₂ | (η₁ | e [ σ₁ ]ˢ) [ σ₂ ]ˢ ≡ (η₁ ⨟ˢ η₂) | e [ η₁ , η₂ | σ₁ ⨟ˢ σ₂ ]ˢ
```

We show the proof only to illustrate where rewriting removes transports. Thus rewriting enables us to concentrate on the gist of the proof, rather than being overwhelmed by transfers. We chose to make passing of the type substitutions η_1 and η_2 explicit. While Agda can infer them in many places, there are equally many places where inference is not possible.

```
Compositionality^SS (` x)      η₁ η₂ σ₁ σ₂ = refl
Compositionality^SS (λx {T₁ = T₁} e) η₁ η₂ σ₁ σ₂ = cong λx (begin
    η₂ | (η₁ | e [ η₁ | σ₁ ⇑ˢ T₁ ]ˢ) [ η₂ | σ₂ ⇑ˢ (T₁ [ η₁ ]ˢ) ]ˢ
  ≡⟨ Compositionality^SS e η₁ η₂ (η₁ | σ₁ ⇑ˢ T₁) (η₂ | σ₂ ⇑ˢ (T₁ [ η₁ ]ˢ)) ⟩ - IH
    (η₁ ⨟ˢ η₂) | e [ η₁ , η₂ | (η₁ | σ₁ ⇑ˢ T₁) ⨟ˢ (η₂ | σ₂ ⇑ˢ (T₁ [ η₁ ]ˢ)) ]ˢ
  ≡⟨ cong ((η₁ ⨟ˢ η₂) | e [_]ˢ) (Lift-Dist-Comp^SS σ₁ σ₂) ⟩
    (η₁ ⨟ˢ η₂) | e [ (η₁ ⨟ˢ η₂) | (η₁ , η₂ | σ₁ ⨟ˢ σ₂) ⇑ˢ T₁ ]ˢ
  □)
Compositionality^SS (Λα e)   η₁ η₂ σ₁ σ₂ = cong Λα (begin
    (η₂ ↑ˢ) | ((η₁ ↑ˢ) | e [ η₁ | σ₁ ⇑ˢ* ]ˢ) [ η₂ | σ₂ ⇑ˢ* ]ˢ
  ≡⟨ Compositionality^SS e (η₁ ↑ˢ) (η₂ ↑ˢ) (η₁ | σ₁ ⇑ˢ*) (η₂ | σ₂ ⇑ˢ*) ⟩ - IH
    ((η₁ ⨟ˢ η₂) ↑ˢ) | e [ (η₁ ↑ˢ) , η₂ ↑ˢ | (η₁ | σ₁ ⇑ˢ*) ⨟ˢ (η₂ | σ₂ ⇑ˢ*) ]ˢ
  ≡⟨ cong (((η₁ ⨟ˢ η₂) ↑ˢ) | e [_]ˢ) (Lift*-Dist-Comp^SS η₁ η₂ σ₁ σ₂) ⟩
    ((η₁ ⨟ˢ η₂) ↑ˢ) | e [ (η₁ ⨟ˢ η₂) | (η₁ , η₂ | σ₁ ⨟ˢ σ₂) ⇑ˢ* ]ˢ
  □)
Compositionality^SS (e₁ · e₂) η₁ η₂ σ₁ σ₂ = cong₂ _·_
  (Compositionality^SS e₁ η₁ η₂ σ₁ σ₂) - IH
  (Compositionality^SS e₂ η₁ η₂ σ₁ σ₂) - IH
Compositionality^SS (e ·* T′) η₁ η₂ σ₁ σ₂ = cong (_·* (T′ [ η₁ ⨟ˢ η₂ ]ˢ))
  (Compositionality^SS e η₁ η₂ σ₁ σ₂) - IH
```

This proof relies on further properties, in particular Lift-Dist-Comp^{SS} (lifting a composition of substitutions is equal to composing their liftings) and Lift*-Dist-Comp^{SS} (its counterpart for lifting over type bindings), where the statement and proof would require explicit transfers. Essentially, these properties form a hierarchy that stops with lemmas for composing two renamings and composing a renaming with a lifting, all of which work similarly. The supplemental material contains full details.

4.2 Transfer Hell: Composition of Substitutions Without Rewriting

We now contrast the development from Section 4.1 with a proof of the same compositionality property, but carried out with explicit transfers. The proof is taken from the supplement of a paper on a logical relation for System F [12]. We aligned the surface syntax of their definitions with ours to enable direct comparison. Throughout this subsection, code reproduced from that prior work is rendered with a shaded background to distinguish it from our own development.

The type of Compositionality^{SS} already exposes the core issue. Because the σ-calculus equations are only propositional, the theorem statement must include subst to bridge indices that are equal but not definitionally equal: the type of the left-hand side of the equality is *not* definitionally equal to the type of the right-hand side. To adapt the types, compositionality of type substitutions is applied to the type of the expression using function *sub*.

```
Compositionality^SS-≡ :
  ∀ {η₁ : Sub n₁ n₂} {η₂ : Sub n₂ n₃}
    {Γ₁ : Ctx n₁} {Γ₂ : Ctx n₂} {Γ₃ : Ctx n₃}
    (e : Expr n₁ Γ₁ T)
    (σ₁ : η₁ | Γ₁ ⇒^S Γ₂) (σ₂ : η₂ | Γ₂ ⇒^S Γ₃) →
  let sub = subst (Expr n₃ Γ₃) (compositionality^SS T η₁ η₂) in
  sub (η₂ | (η₁ | e [ σ₁ ]^S) [ σ₂ ]^S) ≡ (η₁ ⨟^S η₂) | e [ η₁ , η₂ | σ₁ ⨟^S σ₂ ]^S
```

Because subst is part of the statement, every recursive call to the induction hypothesis introduces a fresh application of subst, which has to be discharged in the proof alongside the interesting proof steps. To do so, it must be distributed to the outside of the term and combined with the substs arising from neighboring subterms before the goal can be closed. This process is further complicated by other occurrences of type substitution lemmas that interact non-trivially with the transports. We illustrate the resulting complexity by zooming in on a single case of the inductive proof—type application—of their compositionality lemma.

Firstly, our development already saves one subst in the very definition of substitution application. The type application case requires the swap lemma of type `(T [ T′ ]*) [ η ] ≡ (T [ η ↑^S ]) [ T′ [ η ] ]*` to coerce the term to the expected type, leading to the following definition.

```
η | (_·*_ {T = T} e T′) [ σ ]^S = subst (Expr _ _)
                                    (sym (swap^SS η T T′))
                                    ((η | e [ σ ]^S) ·* (T′ [ η ]^S))
```

Next, we examine the type application case of the compositionality proof itself, a mere application of the inductive hypothesis in Section 4.1. The authors employ heterogeneous equality [10] to avoid some of the administrative reasoning otherwise forced by transfer hell. Without heterogeneous equality, the proof would be even more complex, requiring several auxiliary lemmas that explicitly

shuffle applications of subst around. Thus, the version below already constitutes a streamlined form of the argument.

```
let
  F₂ = Expr n₂ Γ₂ ; E₂ = sym (swap^SS η₁ T T′)          ; sub₂ = subst F₂ E₂
  F₃ = Expr n₃ Γ₃ ; E₃ = sym (swap^SS (η₁ ⨟^S η₂) T T′) ; sub₃ = subst F₃ E₃
  F₅ = Expr n₃ Γ₃ ; E₅ = sym (swap^SS η₂ (T [ η₁ ↑^S ]) (T′ [ η₁ ]^S)) ; sub₅ = subst F₅ E₅
in
R.begin
  η₂ | (η₁ | (e ·* T′) [ σ₁ ]^S) [ σ₂ ]^S
R.≅⟨ refl ⟩                                                    - (1)
  η₂ | (sub₂ ((η₁ | e [ σ₁ ]^S) ·* (T′ [ η₁ ]^S))) [ σ₂ ]^S
R.≅⟨ H.cong₂ {B = Expr n₂ Γ₂} (λ _ ■ → η₂ | ■ [ σ₂ ]^S)        - (2)
       (H.≡-to-≅ (sym E₂))
       (H.≡-subst-removable F₂ E₂ _) ⟩
  η₂ | ((η₁ | e [ σ₁ ]^S) ·* (T′ [ η₁ ]^S)) [ σ₂ ]^S
R.≅⟨ refl ⟩                                                    - (3)
  sub₅ ((η₂ | (η₁ | e [ σ₁ ]^S) [ σ₂ ]^S) ·* (T′ [ η₁ ]^S [ η₂ ]^S))
R.≅⟨ H.≡-subst-removable F₅ E₅ _ ⟩                              - (4)
  (η₂ | (η₁ | e [ σ₁ ]^S) [ σ₂ ]^S) ·* (T′ [ η₁ ]^S [ η₂ ]^S)
R.≅⟨ Hcong₃ {B = λ ■ → Expr n₃ Γ₃ (∀α ■)} {C = λ _ _ → Type n₃ _} - (5)
       (λ _ ■₁ ■₂ → ■₁ ·* ■₂)
       (H.≡-to-≅ (lift-compositionality^SS T η₁ η₂))
       (Compositionality^SS e σ₁ σ₂)
       (H.≡-to-≅ (compositionality^SS T′ η₁ η₂)) ⟩
  ((η₁ ⨟^S η₂) | e [ η₁ , η₂ | σ₁ ⨟^S σ₂ ]^S) ·* (T′ [ η₁ ⨟^S η₂ ]^S)
R.≅⟨ H.sym (H.≡-subst-removable F₃ E₃ _) ⟩                      - (6)
  sub₃ (((η₁ ⨟^S η₂) | e [ η₁ , η₂ | σ₁ ⨟^S σ₂ ]^S) ·* (T′ [ η₁ ⨟^S η₂ ]^S))
R.≅⟨ refl ⟩                                                    - (7)
  (η₁ ⨟^S η₂) | (e ·* T′) [ η₁ , η₂ | σ₁ ⨟^S σ₂ ]^S
R.□
```

The proof proceeds in seven steps.

(1) Unfold the definition of expression substitution at the inner application of σ_1, exposing the subst inside.
(2) Remove the subst that sits inside the term under consideration, justified by heterogeneous equality.
(3) Reveal yet another subst introduced by the substitution application of σ_2
(4) Remove the subst on the outside.
(5) Apply the induction hypothesis while simultaneously coercing the index according to the compositionality law for type substitutions.
(6) Reinsert the subst demanded by the proof goal on the outside.
(7) Fold the definition of expression substitution to conclude the proof.

	With Rewriting		Transfer Hell		Ratio H/R	
	LOC	Chars	LOC	Chars	LOC	Chars
Type substitution + proofs	217	12 246	285	12 584	1.3	1.0
Expression substitution + proofs	199	11 465	1 405	69 846	7.1	6.1
Total	**416**	**23 711**	**1 690**	**82 430**	4.1	3.5
Type-check time	4.49 s		9.51 s		2.1	

Table 2. Proof statistics

Of these, steps (2), (4), the type coercion in (5), and (6), as well as the use of heterogeneous equality, are entirely absent from our rewriting-based proof, only the application of the inductive hypothesis and the implicit (un)folding of definitions remain.

4.3 Quantitative Comparison

We complement the preceding qualitative comparison with a quantitative one. Table 2 records the lines of code and number of characters required to mechanize the compositionality law for expression substitutions, together with the type-checking time of the respective developments. The last column computes the ratios of hell value over rewrite value, i.e., the relative expansion and slowdown of the hell version.

As expected, the amount of code required to formalize type substitution is roughly the same in both developments. For expression substitutions, however, the picture changes: the transfer-hell version requires more than 6–7 times (3–4 counting all substitutions) as much code as the rewriting-based one. As a welcome side effect, the type-checking time is also reduced by about 50% when the σ-calculus equations are installed as rewrite rules.

5 Methodology

5.1 Safety Conditions for Rewriting

Rewrite rules and the reductions of a dependently typed system interact in non-trivial ways [4, 8, 9]. According to RTT's criteria, we can make the equations from Section 2.3 definitional because they preserve subject reduction and consistency.

To uphold subject reduction, the critical property of the newly added rewriting equations is that their *union with the existing reduction relation* is confluent. Agda already provides a standard set of confluent reduction rules: β, η (function and record expansion), δ (definition unfolding), and ι (dependent pattern matching clauses). Agda checks confluence of the extended system using a conservative set of simple syntactic criteria [5].

Some β and ι reductions arising from the functional definitions of renamings and substitutions in Figure 3 interfere with the intended rewrite equalities so that confluence breaks. For this reason, we explicitly hide these definitions inside an opaque block, so that only the equations we install with REWRITE define their reduction behaviour.

Without the opaque definitions, it may happen that the left-hand side of an equation reduces to a normal form, which is different from the right-hand side. Thus, confluence would be violated. As a concrete example, consider the equation η-law (left) in connection with the definition of composition (right):

$$(\mathsf{zero}\ \&^S\ \eta)\ \cdot^S\ (\langle\ \mathsf{wk}^R\ \rangle\ \fatsemi^S\ \eta) \equiv \eta \qquad\qquad (\eta_1\ \fatsemi^S\ \eta_2)\ \alpha = (\eta_1\ \alpha)\ [\ \eta_2\ ]^S$$

Without the opaque block, the definition of composition would δ-unfold the inner application $\langle\, \mathsf{wk}^R \,\rangle \fatsemi^S \eta$ to $\lambda\, \alpha \longrightarrow \eta\, (\mathsf{suc}\, \alpha)$ so that the left-hand side reduces as follows:

$$\begin{aligned}(\mathsf{zero}\, \&^S\, \eta) \cdot^S (\langle\, \mathsf{wk}^R \,\rangle \fatsemi^S \eta) &\equiv (\mathsf{zero}\, \&^S\, \eta) \cdot^S (\lambda\, \alpha \longrightarrow \eta\, (\mathsf{suc}\, \alpha)) \\ &\equiv (\eta\, \mathsf{zero}) \cdot^S (\lambda\, \alpha \longrightarrow \eta\, (\mathsf{suc}\, \alpha))\end{aligned}$$

Two issues arise. First, the definitional reduction happens *before* the rewrite engine tries to match the left-hand side of η-law syntactically. Thus the rewrite never happens[5]. Second, if the rewrite rule were admitted, confluence would fail. The two reducts η (via the rewrite) and $(\eta\, \mathsf{zero}) \cdot^S (\lambda\, \alpha \longrightarrow \eta\, (\mathsf{suc}\, \alpha))$ (via δ-unfolding) are propositionally equal but not definitionally equal. Because the function $\cdot^S$ case-splits on a neutral variable, no further reduction back to η is possible. Hiding the definition of the symbol $\fatsemi^S$ (together with $\cdot^S$ and the other operators) inside an opaque block turns them into stuck symbols, restoring both matchability and confluence.

Consistency is preserved as long as the equalities we rewrite are instantiated with proofs. This is easy to see, because RTT can be translated to extensional type theory (e.g., [6]), a theory known to be consistent.

Agda currently supports all features we require: opaque-defined symbols, proof-instantiated rewrite equations, and triangle-based confluence checking. Rocq currently lacks the first two, although the overall method otherwise transfers.

After internalizing the type-level substitution theory by installing safe rewrites, we aim to extend our method to the expressions level.

5.2 Equational Theory for Expression Substitution

With functoriality for expression substitution in place, one might ask whether we can go further and install a full equational theory for expression substitution as definitional equalities, just as we did at the type level. Indeed, the σ-calculus for type-substitution-indexed expression substitutions is an extension of the one for type substitutions in Section 2.3, and the same holds analogously for expression renamings.

Expression substitutions carry additional operators that have no counterpart at the type level. Every law involving extension or weakening splits into two expression-level laws. One governs the expression variants ($\_\cdot^S\_$, wk^S) and directly mirrors the type-level rule shown in Section 2.3. The other governs the type variants ($\_\cdot^{S*}\_$, wk^{S*}) and is genuinely new. We illustrate this pattern with the η-id law. The expression variant

$$\eta\text{-}\mathsf{Id}^S : \langle\, \mathsf{id}^R \,\rangle \mid (\text{`}\, \mathsf{zero}) \cdot^S (\mathsf{Wk}^S\ T) \equiv \mathsf{Id}^S\ \{\Gamma = \Gamma \triangleright\ T\}$$

is a direct transcription of η-id from Section 2.3, whereas the type variant

$$\eta\text{-}\mathsf{Id}^{S*} : \langle\, \mathsf{wk}^R \,\rangle \mid (\text{`}\, \mathsf{zero}) \cdot^{S*} \mathsf{wk}^{S*} \equiv \mathsf{Id}^S\ \{\Gamma = \Gamma \triangleright^*\}$$

asserts that extending the type-binding weakening with the topmost type variable $\text{`}\, \mathsf{zero}$ yields the identity expression substitution. The same doubling applies to the remaining laws in which extension or weakening appears (η, interact, distributivity and some β laws).

The full equational system is included in the supplement, and Agda verifies its confluence. For the confluence check to go through, we had to artificially block reduction of the traversal functions $\_|\_[\_]^R$ and $\_|\_[\_]^S$ and re-install each of their defining clauses as a rewrite rule. In the type application case, rewriting must perform two reduction steps at once to satisfy the triangle criterion. One reduction normalizes the type index; the other pushes substitution into the expression.

[5]To see why, note that η does not appear in pattern position in the reduct. A precise definition of pattern position is available in the Agda Wiki: https://agda.readthedocs.io/en/latest/language/rewriting.html#general-shape-of-rewrite-rules

5.3 Practical Guidelines for Intrinsic Developments

We have now made the equations of the σ-calculus definitional at two levels of the syntactic hierarchy: at the type level and at the expression level, where the latter depends on the former. In general, this strategy applies to arbitrarily tall towers of intrinsically typed syntaxes, in which each level's type indices themselves support substitution.

More precisely, consider a tower $T_0, T_1, \ldots, T_n$ of syntactic sorts in which the variables and substitutions of T_{i+1} are indexed by the contexts and substitutions of $T_0, \ldots, T_i$. The recipe proceeds bottom-up:

Methodology Recipe. To avoid transfer hell in intrinsically typed towers:

(1) Start at the base sort T_0: define renamings/substitutions and install the σ-calculus equations as rewrite rules.
(2) For each higher sort T_i, define syntax and substitution *on top of* already-definitional lower-level substitutions.
(3) Prove the σ-calculus laws for T_i and install them as rewrite rules.
(4) Check confluence of the combined reduction relation after each level.

The key is that the laws for all levels $< i$ are already definitional, when we define and prove the laws for level i. Otherwise, the types of the level-i operations would no longer match definitionally, and we would be back in transfer hell.

The size of the equational theory grows with each level. A substitution at level i can be lifted under a binder of any sort $T_0, \ldots, T_{i-1}$, and each kind of binder induces its own family of laws, for weakening and extension, and the corresponding interactions with composition and traversal. At the type level this contributes no additional laws. At the expression level we already saw a few new laws, namely the laws governing weakening and extension by types. At level three one would add even more laws, and so on. The rule count therefore grows linearly with the level, but each new family follows the same template as the previous one, so the proof effort per family stays constant.

As an example from the literature, the language System $\mathrm{F}^{\uparrow}_{C}$ [16] features three levels: kinds, types, and expressions. Each level carries its own variables and substitution. Using our approach, it should be possible to construct an intrinsically typed representation for it and establish the substitution theory for all three levels uniformly, without any transfer reasoning.

6 Conclusion

Intrinsically typed syntax facilitates elegant mechanizations by making well-typed programs representable only as well-typed data. For polymorphic calculi this promise is often undermined by pervasive transport reasoning. Our experience with System F shows that the difficulty does not stem from binding structure or indexing itself, but from a mismatch between substitution and definitional equality: substitutions that are extensionally equal fail to compute to a canonical form. By internalizing the equational laws of the σ-calculus as definitional equalities, substitutions become canonical and routine transports disappear from proofs. The resulting developments recover the elegance and concision seen in the simply-typed case while scaling to towers of syntactic sorts. In our System F case study, we observe a decrease of code size by a factor of 3–4 and a speedup by a factor of 2. More broadly, this fresh perspective proposes a practical recipe for mechanizing type systems in dependently typed proof assistants: when intrinsically typed encodings become proof-heavy, a promising strategy is often to strengthen definitional equality rather than introduce additional lemmas or automation.

References

[1] M. Abadi, L. Cardelli, P.-L. Curien, and J.-J. Levy. 1989. Explicit substitutions. In *Proceedings of the 17th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages* (San Francisco, California, USA) *(POPL '90)*. Association for Computing Machinery, New York, NY, USA, 31–46. doi:10.1145/96709.96712

[2] Nick Benton, Chung-Kil Hur, Andrew Kennedy, and Conor McBride. 2012. Strongly Typed Term Representations in Coq. *J. Autom. Reason.* 49, 2 (2012), 141–159. doi:10.1007/S10817-011-9219-0

[3] James Chapman, Roman Kireev, Chad Nester, and Philip Wadler. 2019. System F in Agda, for Fun and Profit. In *Mathematics of Program Construction: 13th International Conference, MPC 2019, Porto, Portugal, October 7–9, 2019, Proceedings* (Porto, Portugal). Springer-Verlag, Berlin, Heidelberg, 255–297. doi:10.1007/978-3-030-33636-3_10

[4] Jesper Cockx. 2020. Type Theory Unchained: Extending Agda with User-Defined Rewrite Rules. In *25th International Conference on Types for Proofs and Programs (TYPES 2019) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 175)*, Marc Bezem and Assia Mahboubi (Eds.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 2:1–2:27. doi:10.4230/LIPIcs.TYPES.2019.2

[5] Jesper Cockx, Nicolas Tabareau, and Théo Winterhalter. 2021. The Taming of the Rew: A Type Theory with Computational Assumptions. *Proc. ACM Program. Lang.* 5, POPL, Article 60 (Jan. 2021), 29 pages. doi:10.1145/3434341

[6] Martin Hofmann. 1997. Syntax and semantics of dependent types. In *Extensional Constructs in Intensional Type Theory*. Springer, 13–54.

[7] Wen Kokke, Jeremy G. Siek, and Philip Wadler. 2020. Programming Language Foundations in Agda. *Sci. Comput. Program.* 194 (2020), 102440. doi:10.1016/J.SCICO.2020.102440

[8] Yann Leray, Gaëtan Gilbert, Nicolas Tabareau, and Théo Winterhalter. 2024. The Rewster: Type Preserving Rewrite Rules for the Coq Proof Assistant. In *15th International Conference on Interactive Theorem Proving (ITP 2024) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 309)*, Yves Bertot, Temur Kutsia, and Michael Norrish (Eds.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 26:1–26:18. doi:10.4230/LIPIcs.ITP.2024.26

[9] Yann Leray and Théo Winterhalter. 2026. Encode the Cake and Eat It Too: Controlling Computation in Type Theory, Locally. *Proc. ACM Program. Lang.* 10, POPL, Article 62 (Jan. 2026), 30 pages. doi:10.1145/3776704

[10] Conor McBride. 2000. Elimination with a Motive. In *Types for Proofs and Programs, International Workshop, TYPES 2000, Durham, UK, December 8-12, 2000, Selected Papers (Lecture Notes in Computer Science, Vol. 2277)*, Paul Callaghan, Zhaohui Luo, James McKinna, and Robert Pollack (Eds.). Springer, 197–216. doi:10.1007/3-540-45842-5_13

[11] Conor McBride. 2005. Type-preserving Renaming and Substitution. (2005). http://strictlypositive.org/ren-sub.pdf Unpublished manuscript.

[12] Hannes Saffrich, Peter Thiemann, and Marius Weidner. 2024. Intrinsically Typed Syntax, a Logical Relation, and the Scourge of the Transfer Lemma. In *Proceedings of the 9th ACM SIGPLAN International Workshop on Type-Driven Development* (Milan, Italy) *(TyDe 2024)*. Association for Computing Machinery, New York, NY, USA, 2–15. doi:10.1145/3678000.3678201

[13] Steven Schäfer, Tobias Tebbi, and Gert Smolka. 2015. Autosubst: Reasoning with de Bruijn Terms and Parallel Substitutions. In *Interactive Theorem Proving*, Christian Urban and Xingyuan Zhang (Eds.). Springer International Publishing, Cham, 359–374. https://www.ps.uni-saarland.de/Publications/documents/SchaeferEtAl_2015_Autosubst_-Reasoning.pdf

[14] Kathrin Stark. 2020. *Mechanising Syntax with Binders in Coq.* Ph. D. Dissertation. Saarland University, Saarbrücken, Germany. https://www.ps.uni-saarland.de/Publications/documents/Stark_2020_Mechanising.pdf

[15] Kathrin Stark, Steven Schäfer, and Jonas Kaiser. 2019. Autosubst 2: Reasoning with Multi-sorted de Bruijn Terms and Vector Substitutions *(CPP 2019)*. Association for Computing Machinery, New York, NY, USA, 166–180. doi:10.1145/3293880.3294101

[16] Brent A. Yorgey, Stephanie Weirich, Julien Cretin, Simon Peyton Jones, Dimitrios Vytiniotis, and José Pedro Magalhães. 2012. Giving Haskell a Promotion. In *Proceedings of the 8th ACM SIGPLAN Workshop on Types in Language Design and Implementation* (Philadelphia, Pennsylvania, USA) *(TLDI '12)*. Association for Computing Machinery, New York, NY, USA, 53–66. doi:10.1145/2103786.2103795